

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

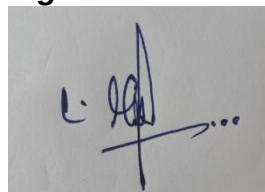
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5

8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
		solutions			
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1)

1. Incorrect Query

Suppose we want to display employee names and their department names.

A common incorrect query is:

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM EMPLOYEE E, DEPARTMENT D;
```

What is the problem?

There is **no condition connecting** EMPLOYEE **and** DEPARTMENT.

Therefore, every employee is combined with every department.

This produces **duplicate/unwanted combinations**.

This is called a **Cartesian Product**.

2. Correct Query Using INNER JOIN

```
SELECT
    E.EmpID,
    E.EmpName,
    D.DeptName
FROM EMPLOYEE E
INNER JOIN DEPARTMENT D
ON E.DeptID = D.DeptID;
```

Why is this correct?

The condition:

```
ON E.DeptID = D.DeptID
```

connects each employee to their corresponding department.

Therefore, only matching records are returned.

3. Example

EMPLOYEE
EmpID EmpName DeptID
101 Arun D1
102 Bala D2
103 Charan D1

DEPARTMENT
DeptID DeptName
D1 IT
D2 HR

Incorrect result

Without the JOIN condition, the database can produce:

```
Arun  IT
Arun  HR
Bala  IT
Bala  HR
Charan IT
Charan HR
```

This is incorrect because employees are matched with departments that they don't belong to.

Correct result

```
Arun  IT
Bala  HR
Charan IT
```

4. If Duplicate Rows Still Occur

Sometimes duplicates occur because the joined table contains multiple matching records.

If the requirement is to display each employee only once, DISTINCT can be used:

```
SELECT DISTINCT
```

```
    E.EmpID,
    E.EmpName,
    D.DeptName
```

```
FROM EMPLOYEE E
```

```
INNER JOIN DEPARTMENT D
```

```
ON E.DeptID = D.DeptID;
```

However, DISTINCT **should not be used simply to hide an incorrect JOIN condition**. The JOIN condition must first be correct.

2)

1. Incorrect Query

Suppose the requirement is:

Find employees whose salary is greater than the average salary of their own department.

An incorrect query is:

```
SELECT E.EmpID, E.EmpName, E.DeptID, E.Salary
```

```

FROM EMPLOYEE E

WHERE E.Salary >

(
    SELECT AVG(E2.Salary)
    FROM EMPLOYEE E2
);

```

Error

The subquery:

```

SELECT AVG(E2.Salary)

FROM EMPLOYEE E2

```

calculates the **average salary of all employees**.

It does **not** calculate the average salary of the employee's department.

The subquery is missing the **correlation condition** connecting E2 with the outer query's E.

2. Correct Correlated Subquery

```

SELECT E.EmpID,
       E.EmpName,
       E.DeptID,
       E.Salary
FROM EMPLOYEE E
WHERE E.Salary >
(
    SELECT AVG(E2.Salary)
    FROM EMPLOYEE E2
    WHERE E2.DeptID = E.DeptID
);

```

Important Correction

This condition creates the correlation:

```

WHERE E2.DeptID = E.DeptID

```

Here:

- E → employee from the **outer query**
- E2 → employee from the **subquery**
- E2.DeptID = E.DeptID → calculates the average salary **only within the same department**

Example EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Arun	D1	40000
102	Bala	D1	60000
103	Charan	D2	30000
104	Deepak	D2	50000

Department averages:

$$D1 = (40000 + 60000) / 2 = 50000$$

$$D2 = (30000 + 50000) / 2 = 40000$$

The correct query returns:

EmpID	EmpName	DeptID	Salary
102	Bala	D1	60000
104	Deepak	D2	50000

3)

1. Incorrect View

Suppose the requirement is to create a view showing **department-wise total employees and average salary**.

An incorrect view could be:

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AverageSalary
FROM DEPARTMENT D
JOIN EMPLOYEE E
ON D.DeptID = E.DeptID;
```

Error

The query uses aggregate functions:

COUNT()
AVG()

but does **not use** GROUP BY for DeptName.

Therefore, the view definition can produce an error because DeptName is a non-aggregated column.

2. Correct View

Add GROUP BY D.DeptName:

```
CREATE VIEW DeptSalaryView AS
SELECT
    D.DeptName,
    COUNT(E.EmpID) AS TotalEmployees,
    AVG(E.Salary) AS AverageSalary
FROM DEPARTMENT D
INNER JOIN EMPLOYEE E
    ON D.DeptID = E.DeptID
GROUP BY D.DeptName;
```

3. Query the View

```
SELECT *
FROM DeptSalaryView;
```

4. Optimization

A good optimization is to group using the department's unique identifier as well as its name:

```
CREATE VIEW DeptSalaryView AS
SELECT
    D.DeptID,
    D.DeptName,
    COUNT(E.EmpID) AS TotalEmployees,
    AVG(E.Salary) AS AverageSalary
FROM DEPARTMENT D
INNER JOIN EMPLOYEE E
    ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;
```

This is preferable because DeptID uniquely identifies each department.

Explanation

Error

The original view uses:

```
COUNT()
AVG()
```

with DeptName, but there is no GROUP BY.

Correction

Use:

```
GROUP BY D.DeptID, D.DeptName
```

Optimization

- Use explicit INNER JOIN.
- Group by the department's key.
- Select only the required columns.
- Use meaningful aliases such as TotalEmployees and AverageSalary.

4)

1. Relations

Consider these two relations:

STUDENT(SID, StudentName, DeptID)

DEPARTMENT(DepartmentID, DeptName)

Both relations contain the common attribute:

DeptID

2. Incorrect Expression

Suppose the incorrect expression is:

STUDENT $\times$ DEPARTMENT

Error

$\times$ represents the **Cartesian Product**, not a natural join.

It combines every student with every department, producing incorrect and duplicate combinations.

3. Correct Natural Join

The correct relational algebra expression is:

STUDENT $\bowtie$ DEPARTMENT

Since both relations have the common attribute DeptID, the **natural join automatically joins matching DeptID values**.

4. Equivalent Expression Using Condition

The natural join can also be expressed using a Cartesian product followed by selection:

$\sigma_{\text{STUDENT.DeptID} = \text{DEPARTMENT.DeptID}}(\text{STUDENT} \times \text{DEPARTMENT})$

This means:

1. First create the Cartesian product.
2. Select only rows where STUDENT.DeptID = DEPARTMENT.DeptID.

Example

STUDENT

SID	StudentName	DeptID
101	Arun	D1
102	Bala	D2
103	Charan	D1

DEPARTMENT

DeptID	DeptName
D1	IT
D2	CSE

Correct Natural Join

STUDENT $\bowtie$ DEPARTMENT

Output

SID	StudentName	DeptID	DeptName
101	Arun	D1	IT
102	Bala	D2	CSE
103	Charan	D1	IT

5)

1. What is Referential Integrity?

Referential integrity ensures that a **foreign-key value in a child table must match an existing primary-key value in the parent table.**

For example:

DEPARTMENT(DeptID) $\rightarrow$ EMPLOYEE(DeptID)
Parent Child

2. Incorrect Situation

Suppose we have:

DEPARTMENT

DeptID	DeptName
D1	IT
D2	HR

Now we try to insert an employee with a department that does not exist:

```
INSERT INTO EMPLOYEE  
(EmpID, EmpName, DeptID) VALUES  
(101, 'Arun', 'D5');
```

Problem

D5 does not exist in the DEPARTMENT table.

Therefore, if EMPLOYEE.DeptID is a foreign key, the database must **reject the insertion.**

This is a **referential integrity violation.**

3. Correct Table Definitions

First create the parent table:

```
CREATE TABLE DEPARTMENT
(
  DeptID VARCHAR(10) PRIMARY KEY,
  DeptName VARCHAR(50) NOT NULL
);
```

Then create the child table:

```
CREATE TABLE EMPLOYEE
(
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50) NOT NULL,
  DeptID VARCHAR(10),

  FOREIGN KEY (DeptID)
  REFERENCES DEPARTMENT(DeptID)
);
```

4. Insert Valid Department

```
INSERT INTO DEPARTMENT VALUES ('D1', 'IT');
INSERT INTO DEPARTMENT VALUES ('D2', 'HR');
```

5. Insert Valid Employee

```
INSERT INTO EMPLOYEE VALUES (101, 'Arun', 'D1');
```

This is accepted because:

D1 exists in DEPARTMENT.

6. Demonstrate Referential Integrity Violation

```
INSERT INTO EMPLOYEE VALUES (102, 'Bala', 'D5');
```

This is rejected because:

D5 does not exist in DEPARTMENT.

Therefore, the **FOREIGN KEY constraint prevents invalid data.**

7. Correct the Invalid Statement

Instead of using a non-existing department:

```
INSERT INTO EMPLOYEE VALUES (102, 'Bala', 'D5');
```

Use an existing department:

```
INSERT INTO EMPLOYEE VALUES (102, 'Bala', 'D2');
```

Now the insertion succeeds because D2 exists in DEPARTMENT.

6)

Scenario

Suppose we have:

```
EMPLOYEE(EmpID, EmpName, DeptID, Salary)
```

```
DEPARTMENT(DeptID, DeptName)
```

We want to display employees along with their department names.

2. Inefficient Query Using Nested Subquery

```
SELECT EmpID, EmpName,  
       (SELECT DeptName  
        FROM DEPARTMENT D  
        WHERE D.DeptID = E.DeptID) AS DeptName  
FROM EMPLOYEE E;
```

Problem

The inner query is evaluated for employee records to find the matching department.

This can be inefficient for a large number of employees because the database repeatedly searches the DEPARTMENT table.

3. Optimized Query Using JOIN

```
SELECT  
  E.EmpID,  
  E.EmpName,  
  D.DeptName  
FROM EMPLOYEE E INNER JOIN DEPARTMENT D ON E.DeptID = D.DeptID;
```

Why is this better?

The JOIN directly connects the two related tables using:

```
E.DeptID = D.DeptID
```

The database optimizer can choose an efficient execution strategy for the join.

4. Example

EMPLOYEE

EmpID	EmpName	DeptID
101	Arun	D1
102	Bala	D2
103	Charan	D1

DEPARTMENT

DeptID	DeptName
D1	IT
D2	HR

Output

EmpID	EmpName	DeptName
101	Arun	IT
102	Bala	HR
103	Charan	IT

5. Further Optimization with Indexes

For large tables, indexes on the columns used for joining can improve lookup performance.

```
CREATE INDEX idx_employee_dept
```

```
ON EMPLOYEE(DeptID);
```

```
CREATE INDEX idx_department_dept
```

```
ON DEPARTMENT(DeptID);
```

Since DEPARTMENT.DeptID is a primary key, many DBMSs already create an index for it automatically.

7)

1. Scenario

Suppose we have a SALES table:

```
SALES(SaleID, Category, ProductName, Amount)
```

We want to calculate **total sales for each category**.

2. Incorrect Query

```
SELECT Category, ProductName, SUM(Amount) AS TotalSales
FROM SALES
GROUP BY Category;
```

Error

The query selects:

ProductName

but ProductName is **not included in the** GROUP BY **clause** and is also not inside an aggregate function.

Therefore, the query is invalid in standard SQL and may produce an error such as:

Column 'ProductName' must appear in the GROUP BY clause

3. Correct Query

If we need **total sales per category**, ProductName should be removed:

```
SELECT
```

```
    Category,
```

```
    SUM(Amount) AS TotalSales
```

```
FROM SALES
```

```
GROUP BY Category;
```

This correctly calculates the total sales for each category.

4. If We Need Product-Wise Sales

If the requirement is to display both category and product, then both columns must be grouped:

```
SELECT
```

```
    Category,
```

```
    ProductName,
```

```
    SUM(Amount) AS TotalSales
```

```
FROM SALES
```

```
GROUP BY Category, ProductName;
```

Example

SALES

SaleID	Category	ProductName	Amount
--------	----------	-------------	--------

1	Electronics	Laptop	50000
---	-------------	--------	-------

2	Electronics	Mouse	1000
---	-------------	-------	------

3	Furniture	Chair	5000
---	-----------	-------	------

4	Furniture	Table	8000
---	-----------	-------	------

Correct category-wise output

Electronics	51000
-------------	-------

Furniture	13000
-----------	-------

8)

1. Scenario

Suppose we have:

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

We want to increase the salary of **only employee 101 by 10%**.

2. Incorrect Query

```
UPDATE EMPLOYEE  
SET Salary = Salary * 1.10;
```

Error

The query has **no WHERE condition**.

Therefore, the UPDATE statement applies to **every row** in the EMPLOYEE table.

So, the salary of **all employees** is increased by 10%.

3. Correct Query

```
UPDATE EMPLOYEE  
  
SET Salary = Salary * 1.10  
  
WHERE EmpID = 101;
```

Now, only employee 101 is updated.

4. Example

Before Update

EmpID	EmpName	Salary
101	Arun	40000
102	Bala	50000
103	Charan	60000

If we execute the incorrect query:

```
UPDATE EMPLOYEE
```

SET Salary = Salary * 1.10; all three salaries are changed:

EmpID	Salary
101	44000
102	55000
103	66000

This is **not intended** if only Arun should receive the increase.

5. Correct Result

Using:

```
UPDATE EMPLOYEE
```

```
SET Salary = Salary * 1.10
```

```
WHERE EmpID = 101;
```

the result is:

EmpID	EmpName	Salary
101	Arun	44000
102	Bala	50000
103	Charan	60000

Only Arun's salary is modified.

9)

1. Relations

Assume the following tables:

CUSTOMER(CustomerID, CustomerName)

ORDERS(OrderID, CustomerID, ProductID)

PRODUCT(ProductID, ProductName, Price)

We want to display the **customer name, order ID, product name, and price.**

2. Inefficient Query Using Subqueries

```
SELECT
  C.CustomerName,
  O.OrderID,
  (SELECT P.ProductName
   FROM PRODUCT P
   WHERE P.ProductID = O.ProductID) AS ProductName,
  (SELECT P.Price
   FROM PRODUCT P
   WHERE P.ProductID = O.ProductID) AS Price
FROM CUSTOMER C WHERE C.CustomerID IN
(
  SELECT O2.CustomerID
  FROM ORDERS O2
);
```

Problems

1. It uses nested subqueries unnecessarily.
2. The PRODUCT table is searched repeatedly.

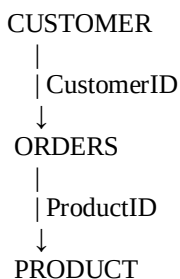
3. The IN subquery is unnecessary because we can directly join CUSTOMER and ORDERS.
4. The query is harder to read and optimize.

3. Optimized Query Using JOIN

```
SELECT
  C.CustomerName,
  O.OrderID,
  P.ProductName,
  P.Price
FROM CUSTOMER C
INNER JOIN ORDERS O
  ON C.CustomerID = O.CustomerID
INNER JOIN PRODUCT P
  ON O.ProductID = P.ProductID;
```

Why is this better?

The query directly connects the three related tables:



This avoids unnecessary nested subqueries and gives the database optimizer more flexibility in choosing an efficient execution plan.

4. Appropriate Indexes

Indexes should be created on columns frequently used for **JOINS**.

Index on ORDERS.CustomerID

```
CREATE INDEX idx_orders_customer ON ORDERS(CustomerID);
```

Index on ORDERS.ProductID

```
CREATE INDEX idx_orders_product ON ORDERS(ProductID);
```

CUSTOMER.CustomerID and PRODUCT.ProductID are primary keys, so they are generally already indexed.

5. Example Output

CustomerName	OrderID	ProductName	Price
Arun	O101	Laptop	50000
Bala	O102	Mouse	800
Charan	O103	Keyboard	1200

1. Scenario

Suppose we want to find **students who are enrolled in the DBMS course**.

Relations:

STUDENT(SID, StudentName)

ENROLLMENT(SID, CID)

COURSE(CID, CourseName)

2. Incorrect Expression

A common logical error is using **FOR ALL** when we actually need to check whether **there exists** a matching DBMS course.

```
{ S |  
  STUDENT(S) AND  
  FOR ALL C (  
    COURSE(C) AND C.CourseName = 'DBMS'  
    AND EXISTS E (  
      ENROLLMENT(E) AND  
      E.SID = S.SID AND  
      E.CID = C.CID  
    )  
  )  
}
```

Problem

The quantifier FOR ALL C is too restrictive.

It requires the condition to be satisfied for **every tuple in COURSE**, rather than simply finding a DBMS course that the student is enrolled in.

This can result in an **empty or incorrect result**.

3. Correct Expression

Use the **EXISTS** quantifier because we need to find whether there is a DBMS course that the student has enrolled in.

```
{ S |  
  STUDENT(S) AND  
  EXISTS C (  
    COURSE(C) AND  
    C.CourseName = 'DBMS' AND  
    EXISTS E (  
      ENROLLMENT(E) AND  
      E.SID = S.SID AND  
      E.CID = C.CID  
    )  
  )  
}
```

4. Explanation

STUDENT(S)

Identifies S as a tuple from the STUDENT relation.

EXISTS C

Checks whether there is **at least one course** satisfying the required condition.

C.CourseName = 'DBMS'

Ensures that the course is DBMS.

EXISTS E

Checks whether an enrollment record exists.

E.SID = S.SID

Connects the enrollment to the student.

E.CID = C.CID

Connects the enrollment to the DBMS course.

Therefore, the expression returns students who are enrolled in DBMS.

Example

STUDENT

SID StudentName

101 Arun

102 Bala

103 Charan

COURSE

CID CourseName

C1 DBMS

C2 OS

ENROLLMENT

SID CID

101 C1

102 C2

103 C1

Output

Arun

Charan

These students have an enrollment record for the DBMS course.